

Lernziele:

- Aus Expression-Trees Assembler-Code erzeugen
- Wie in jedem Entwicklungszyklus: Code Refactoring
- Inkrementelle Softwareentwicklung und Vertiefung des bisher Gelernten

Schritt 1 (Parser: Code Review)

Seit der letzten Session unterstützt unser Parser folgende Grammatik:

```
expr = term { ( "+" | "-" ) term }
term = factor { ( "*" | "/" ) factor }
factor = decimal-literal | "(" expr ")"
```

Bemerkung: Stellt sicher, dass die Namen der Parse-Funktionen konsistent mit der Grammatik sind. Diese sollten `parseExpr`, `parseTerm` und `parseFactor` heißen. Alternativ könnt ihr natürlich auch die Bezeichnungen in der Grammatik anpassen. Wichtig ist, dass Grammatik und Implementierung konsistent sind.

Schritt 2 (Parser: Code Refactoring)

Bei mir sieht die Implementierung von `parseTerm` aktuell so aus:

```
28 fn parseTerm(): -> Expr
29 {
30     local left: -> Expr = parseFactor();
31     if (!left) {
32         return nullptr;
33     }
34     while (token.kind == ASTERISK || token.kind == SLASH) {
35         local tok: Token = token;
36         getToken();
37         local right: -> Expr = parseFactor();
38         if (!right) {
39             printf("%u.%u: factor expected\n", token.pos.row, token.pos.col);
40             return nullptr;
41         }
42         local exprKind: ExprKind = tok.kind == ASTERISK ? EXPR_MUL : EXPR_DIV;
43         left = createBinaryExpr(exprKind, left, right);
44     }
45     return left;
46 }
```

Der Code für `parseExpr` sieht ähnlich aus. Was mich in beiden Fällen stört, ist, wie die Art eines Tokens (`ASTERISK` oder `SLASH`) auf die zugehörige Knotenart einer Expression abgebildet wird. Hier wird zum Beispiel in Zeile 34 erst in einer lokalen Variable das Token mit dem Operator in einer Variable `tok` abgespeichert, dann wird in Zeile 41 abhängig von `tok` eine Variable `exprKind` mit der zugehörigen Knotenart für die Expression initialisiert. Diese zwei Dinge gehören zusammen und sind für meinen Geschmack deshalb zu weit voneinander entfernt, und das führt aus meiner Erfahrung früher oder später zu Problemen.

Um den Code robuster zu gestalten, kann man eine (interne) Hilfsfunktion `exprKindFromToken` schreiben, die zu einem Token die passende Knotenart liefert (oder mit einer Assertion das Programm abbricht). Ergänzt dazu folgende Funktion und verwendet sie in `parseTerm` und `parseExpr`:

```
fn exprKindFromToken(token: Token): ExprKind
{
    switch (token.kind) {
        case ASTERISK: return EXPR_MUL;
        // ...
        default:
            assert(0 && "internal error: exprKindFromToken");
            return 0;
    }
}
```

Bemerkung: Ihr werdet feststellen, dass die Funktionen `parseTerm` und `parseExpr` nun bis auf wenige Stellen gleich aussehen. Wir werden das später ausnutzen, um den Code noch weiter zu vereinfachen, indem wir daraus eine einzige Funktion machen. Der Vorteil wird dann sein, dass man mit wenig Änderungen weitere (Binär-)Operatoren in der Grammatik unterstützen kann.

Vorbemerkung für die weiteren Schritte

Das Ziel ist, dass aus den Expression-Trees, die der Parser erzeugt, Assembler-Code generiert werden kann. In der ersten Version des Parsers hat dieser direkt den Assembler-Code erzeugt. Wir haben den Code in gewisser Hinsicht zwar verbessert, aber die Funktionalität verringert. Wir versuchen jetzt also, das wieder zu unterstützen, was früher schon einmal geklappt hat, aber mit einem besseren Design, das leichter zu erweitern und zu warten ist.

Aktuell gibt es die Funktion `printExprTree`, mit der man aus einem Expression-Tree Latex-Code erzeugen kann. Analog wird es eine Funktion `loadExpr` geben, die Assembler-Code erzeugt. Um diese Funktion "elegant" implementieren zu können, wird jetzt aber etwas Vorarbeit notwendig sein. In `gen.hdr` soll ein abstraktes Interface für die Code-Generierung zur Verfügung gestellt werden, das man später für `loadExpr` verwenden kann.

Schritt 3 (Interface für den Code-Generator und ein Testprogramm)

Man muss sich zunächst klar machen, dass der Code-Generator im Wesentlichen nur Funktionen implementiert, um Text auszugeben. Nur dass dieser Text eben Assembler-Code darstellt. Es wird zum Beispiel Funktionen geben, die Assembler-Code ausgeben, mit dem eine Konstante in ein Register geladen werden kann, oder mit dem Register addiert werden können. Zusätzlich erlaubt es der Code-Generator aber auch zu verwalten, welche Register aktuell zur Verfügung stehen.

Aber schauen wir uns zunächst nur die neue Version von `gen.hdr` an:

```
1 type GenReg: unsigned;
2
3 extern fn genReleaseReg(reg: GenReg);
4
5 extern fn genConstant(val: -> char): GenReg;
6
7 enum GenInstr
8 {
9     GEN_ADD,
10    GEN_SUB,
11    GEN_MUL,
12    GEN_DIV,
13 };
14
15 extern fn genInstr(instr: GenInstr, left: GenReg, right: GenReg): GenReg;
```

In Zeile 1 wird der Typ `GenReg` als Integer-Typ vereinbart. Das kann sich später noch ändern, sodass `GenReg` zum Beispiel ein Pointer-Typ oder Struct-Typ sein wird. Im Zusammenspiel mit dem Code-Generator sollte man also immer `GenReg` verwenden und sich nicht auf die aktuelle Vereinbarung in Zeile 1 verlassen.

Die Bedeutung der Funktion `genReleaseReg` möchte ich erst nach der Bedeutung von `genConstant` und `genInstr` genauer erklären:

- Mit `genConstant` kann man Assembler-Code erzeugen, der eine Konstante in ein Register lädt. Der Aufruf von `genConstant("123")` könnte zum Beispiel folgenden Text ausgeben:

```
load 123, %1
```

also Assembler-Code, um die Konstante 123 in Register `%1` zu laden. Die Funktion gibt außerdem das Ziel-Register zurück, also in diesem Fall den Wert `1`.

- Mit `genInstr` kann man Assembler-Code erzeugen, um Register zu addieren, zu subtrahieren, zu multiplizieren oder zu dividieren. Die Art der Operation kann durch eine entsprechende

Enum-Konstante angegeben werden. Der Aufruf von `genInstr(GEN_ADD, 1, 2)` könnte folgenden Text ausgeben:

```
add %1, %2, %3
```

und auch diese Funktion gibt das Ziel-Register zurück, also in diesem Fall den Wert 3.

Intern wird im Code-Generator verwaltet, welche Register bei der Erzeugung von Assembler-Code bereits verwendet wurden, und ein bereits verwendetes Register wird nicht nochmals verwendet, es sei denn, es wird mit der Funktion `genReleaseReg` freigegeben.

Ein einfaches Testprogramm, das sich nicht darauf verlässt, dass Register mit einem Integer repräsentiert werden, könnte so aussehen:

```
1 @ "gen.hdr"
2
3 fn main()
4 {
5     local r1: GenReg = genConstant("12");
6     local r2: GenReg = genConstant("34");
7     local r3: GenReg = genInstr(GEN_ADD, r1, r2);
8     genReleaseReg(r3);
9     r3 = genInstr(GEN_SUB, r1, r2);
10    genReleaseReg(r1);
11    genReleaseReg(r2);
12    r1 = genConstant("42");
13    genReleaseReg(r1);
14 }
```

In Zeile 5 soll Code erzeugt werden, um den Wert 12 in ein Register zu laden, und in Zeile 6 Code, um den Wert 34 in ein anderes Register zu laden. In Zeile 7 soll Code erzeugt werden, um diese Register zu addieren und in ein weiteres, noch unbenutztes Register zu laden. In Zeile 8 wird das letzte Register (mit dem Ergebnis der Addition) wieder freigegeben und steht damit in Zeile 9 wieder zur Verfügung, wenn eine weitere Instruktion erzeugt wird. Und so weiter.

Aufgabe: Schreibt `gen.hdr` und `xtest_gen.abc`. Die Implementierung von `gen.abc` erfolgt erst im nächsten Schritt, ihr könnt also noch keine ausführbare Datei erzeugen. Aber ihr könnt mit

```
abc -c xtest_gen.abc
```

bereits ein Objekt-File erzeugen und damit überprüfen, dass es keine Tippfehler oder vergessene Includes gibt.

Schritt 4 (Implementierung des Code-Generator)

In `gen.abc` gibt es zusätzlich zu den im Header deklarierten Funktionen noch ein globales Array `usedReg` und eine interne Funktion `getReg`. Mit dem Array wird festgehalten, welche Register bereits benutzt wurden, und mit `getReg` kann man ein unbenutztes Register anfordern. Das Array ist ein simples Array von Booleans. Ist ein Eintrag `true`, dann ist es in Benutzung, und ansonsten steht es zur Verfügung. Mit `getReg` bekommt man das erste Register größer Null (da Register 0 eine Sonderbedeutung hat), das zur Verfügung steht. Mit `genReleaseReg` wird ein Array-Eintrag einfach auf `false` gesetzt. Eine einfache Implementierung kann also so aussehen:

```

1 @ <stdio.h>
2
3 @ "gen.hdr"
4
5 global usedReg: array[16] of bool;
6
7 fn getReg(): GenReg
8 {
9     for (local r: GenReg = 1; r < sizeof(usedReg)/sizeof(usedReg[0]); ++r) {
10         if (!usedReg[r]) {
11             usedReg[r] = true;
12             return r;
13         }
14     }
15     assert(0 && "getReg: out of registers!\n");
16     return 0;
17 }
18
19 fn genReleaseReg(reg: GenReg)
20 {
21     assert(usedReg[reg]);
22     usedReg[reg] = false;
23 }

```

Zur Verfügung stehen insgesamt die Register `%1` bis `%15`. Mit geeigneten Assertions wird hier noch überprüft, dass `getReg` ein freies Register gefunden hat und in `genReleaseReg` auch tatsächlich ein benutztes Register freigegeben wird.

Das war der komplizierteste Teil. Die weiteren Funktionen generieren im Wesentlichen nur Text und geben ein zuvor mit `getReg` allokiertes Zielregister zurück. Zum Beispiel kann `genConstant` so implementiert werden:

```

25 fn genConstant(val: -> char): GenReg
26 {
27     local reg: GenReg = getReg();
28     printf("\tload %s, %%d\n", val, reg);
29     return reg;
30 }

```

Aufgabe

Implementiert die Funktion `genInstr`. Dabei sollen alle möglichen Werte von `instr` abgedeckt werden:

- Der Aufruf von `genInstr(GEN_ADD, r1, r2)` sollte Assembler-Code der Form
`add %1, %2, %3`
- Der Aufruf von `genInstr(GEN_SUB, r1, r2)` sollte Assembler-Code der Form
`sub %1, %2, %3`
- Der Aufruf von `genInstr(GEN_MUL, r1, r2)` sollte Assembler-Code der Form
`mul %1, %2, %3`
- Der Aufruf von `genInstr(GEN_DIV, r1, r2)` sollte Assembler-Code der Form
`div %1, %2, %3`

erzeugen. Testet die Implementierung mit `xtest_gen`. Hier kann und sollte man den Test eigentlich anpassen, so dass alle Operationen ausgeführt werden. Ich selbst wäre dafür aber jetzt wahrscheinlich zu faul und würde mit dem nächsten Schritt weitermachen und Copy-Paste-Fehler erst dann korrigieren, wenn mir diese im nächsten Schritt auffallen.

Schritt 5 (Code aus Expression-Trees erzeugen)

In `expr.hdr` deklarieren wir analog zu `printExprTree` eine Methode `loadExpr`. Diese Methode erwartet einen Zeiger auf einen Expression-Tree und gibt ein Register zurück:

```
// methods
extern fn printExprTree(expr: ->Expr);
extern fn loadExpr(expr: ->Expr): GenReg;
```

Zweck der Funktion ist, dass Assembler-Code erzeugt wird, mit dem die Expression in ein (freies) Register geladen werden kann. Um das zu testen, können wir bereits das Testprogramm `xtest_expr.abc` anpassen und an geeigneter Stelle (nachdem der Expression-Tree erzeugt wurde und bevor er wieder freigegeben wurde) diese Zeile einfügen:

```
genReleaseReg(loadExpr(expr));
```

Bevor ihr euch an die Implementierung der Methode macht, überprüft erst, ob ihr für das Testprogramm ein Objekt-File erzeugen könnt. Eventuell habt ihr ja Tippfehler oder die eine oder andere vergessene Include-Direktive.

Schritt 6 (Implementierung von loadExpr)

Für die Implementierung von `loadExpr` kann man die Funktion `printExprTree` kopieren, den Namen ändern und das Ganze etwas anpassen. Zum Beispiel gibt man bei einem Primary-Node nun nicht mehr den Wert aus, sondern verwendet `genConstant`, um Assembler-Code auszugeben, der den Wert in ein Register lädt. Das Zielregister, das man von `genConstant` erhält, gibt man dann zurück. Wie bei `printExprTree` wird der ganze Expression-Tree rekursiv durchlaufen. Die Struktur sieht also so aus:

```
56 fn loadExpr(expr: ->Expr): GenReg
57 {
58     if (expr->kind >= EXPR_PRIMARY && expr->kind <= EXPR_PRIMARY_END) {
59         return genConstant(expr->decimal);
60     } else if (expr->kind >= EXPR_BINARY && expr->kind <= EXPR_BINARY_END) {
61         // TODO ...
62     }
63     assert(0 && "loadExpr: not all cases handled!");
64     return genConstant("0");
65 }
```

Wird Zeile 63 erreicht, wurde oben ein Fall nicht abgedeckt, was bei Test dann einen Assertion-Failure auslöst. Die Zeile 64 wird also nie erreicht; dass man hier dennoch etwas zurückgibt, hat nur den Zweck, dass der Compiler nicht meckert, dass die Funktion nicht in jedem Fall einen Rückgabewert hat.

Die fehlenden Fälle für Binary-Expressions könnt ihr nun nach diesem Muster programmieren:

1. Mit `loadExpr` wird Assembler-Code erzeugt, mit dem der linke Child-Node in ein Register geladen wird. Den Rückgabewert kann man in einer lokalen Variable `left` speichern.
2. Mit `loadExpr` wird Assembler-Code erzeugt, mit dem der rechte Child-Node in ein Register geladen wird. Den Rückgabewert kann man in einer lokalen Variable `right` speichern.
3. Die Register `left` und `right` kann man jetzt mit `genReleaseReg` wieder zur Verfügung stellen, sodass zum Beispiel bei einer Addition eines dieser Register als Zielregister verwendet werden kann.
4. Dann erzeugt man abhängig von der Expression-Art Assembler-Code mit `genInstr` und gibt den erhaltenen Rückgabewert zurück.

Testet die Implementierung dann mit `xtest_expr`.

Schritt 7 (Jetzt fügt sich alles zusammen)

In `xtest_parser.abc` könnt ihr jetzt nach dem Aufruf von `printExprTree` auch Assembler-Code für die Expression erzeugen.

Schritt 8 (Code-Generator erweitern)

Um für die ULM ein funktionierendes Programm erzeugen zu können sollten wir auch Code für eine Halt-Instruktion erzeugen können. Das ist eine gute Übung für das Erweitern von Code in einem Projekt, also insbesondere der methodischen Vorgehensweise.

Aufgabe

In `gen.hdr` soll `genHalt` deklariert werden:

```
extern fn genHalt(exitCode: GenReg);
```

In `gen.abc` soll die implementiert werden und in `xtest_parser` soll am Ende eine Halt-Instruktion erzeugt werden bei der als Exit-Code der Wert der Expression verwendet wird.

Schritt 9 (Eigenes Programm für den Compiler)

Das Programm `xtest_parser` gibt jetzt sowohl Latex-Code als auch Assembler-Code aus. Das stört, denn jetzt kann man die Ausgabe weder für Latex noch den Assembler verwenden. Kopiert `xtest_parser.abc` nach `xtest_abc.abc`. Bei `xtest_parser.abc` entfernt die Generierung des Assembler-Codes und in `xtest_abc.abc` die Generierung von Latex-Code.

Wenn man jetzt `xtest_abc` ausführt und `3 + 5 * 6` eingibt erhält man folgenden Assembler-Code:

```
load 3, %1
load 5, %2
load 6, %3
mul %2, %3, %2
add %1, %2, %1
halt %1
```

Die Ausgabe kann man in eine Datei `foo.s` umlenken, wenn man das Programm wie folgt ausführt:

```
./xtest_abc > foo.s
```

Mit dem Ulm-Assembler `ulmas` kann man daraus ein `a.out` erzeugen, dann mit `ulm a.out` ausführen und mit `echo $?` den Exit-Code abfragen. Allerdings unterstützt weder unsere Hardware noch der Assembler die Operationen Subtraktion und Division!

Schritt 10 (ISA erweitern)

Fügt diese Zeilen in `simple.isa` hinzu:

```
0x14 R_R_R
:  sub %x, %y, %z
    ulm_sub64(ulm_regVal(x), ulm_regVal(y), z);

0x18 R_R_R
:  div %x, %y, %z
    ulm_idiv64(ulm_regVal(x), ulm_regVal(y), z);
```

Mit `make` wird die Hardware und der Assembler neu gebaut und sollte dann Subtraktion und Division unterstützen.